

Traffic Engineering with MPLS

Antonio Carzaniga
Pasquale Polverino

26 May 2025

We want to engineer traffic in a network so as to optimize the overall goodput. This means that we want to maximize the amount of data transferred by applications per unit time. The problem is that routing is traditionally based on shortest paths. This favors latency over throughput.

As an analogy, consider car traffic in a city. Now, car traffic is already non-uniform in time and space, since many people live in the suburbs, and commute to the center in the morning, and back out in the afternoon. However, imagine for simplicity that everyone starts at random times, and goes from a random location to another random location. Still, if everyone chooses the shortest path, then the city center will be congested. Intuitively, this is because shortest paths are close to a straight line between source and destinations, and for most source–destination pairs, that straight-line route goes through the center.

In other words, shortest-path routing tends to route many flows through the same links that are most favorable for latency. Those then become congested.

In summary, traffic engineering is about spreading traffic more uniformly over the network, so as to use as many links as possible.

Basic Experimental Setup: Link Transmission Rates

We start experimenting with the usual network emulation based on name spaces (Mininet), limiting the link capacities. In particular, we start by limiting all router-to-router links to a fixed maximal capacity T . Here is a script to setup our example network with a routing scheme based on the shortest path using Mininet:

```
#!/usr/bin/env python

from mininet.topo import Topo
from mininet.link import TCLink
from mininet.net import Mininet
from mininet.node import Node
from mininet.log import setLogLevel, info
from mininet.cli import CLI
from mininet.nodelib import LinuxBridge # requires bridge-utils

class LinuxRouter(Node):
    def config(self, **params):
        super(LinuxRouter, self).config(**params)
        # Enable forwarding on the router
        self.cmd("sysctl net.ipv4.ip_forward=1")
```

```

def terminate(self):
    self.cmd("sysctl net.ipv4.ip_forward=0")
    super(LinuxRouter, self).terminate()

class BasicTopo(Topo):
    def build(self, **_opts):
        for i in [1, 2, 3, 4]:
            self.addNode("R%d" % i, cls=LinuxRouter, ip=None)

        host_lans = {
            1: [1, 2],
            4: [3, 4],
        }

        for i, hosts in host_lans.items():
            switch = self.addSwitch("S%d" % i)

            self.addLink(
                switch,
                "R%d" % i,
                intfName2="R%d-eth0" % i,
                params2={"ip": "10.0.%d.254/24" % i},
            )

            for idx, host in enumerate(hosts):
                self.addHost("H%d" % host, ip=None, defaultRoute="via 10.0.%d.254" % i)
                self.addLink(
                    "H%d" % host,
                    switch,
                    intfName1="H%d-eth0" % host,
                    params1={"ip": "10.0.%d.%d/24" % (i, idx + 1)},
                )

        for u, v in [(1, 2), (1, 3), (2, 3), (2, 4), (3, 4)]:
            self.addLink(
                "R%d" % u,
                "R%d" % v,
                intfName1="R%d-eth%d" % (u, v),
                params1={"ip": "10.%d.%d.%d/24" % (u, v, u)},
                intfName2="R%d-eth%d" % (v, u),
                params2={"ip": "10.%d.%d.%d/24" % (u, v, v)},
                bw=1,
                use_tbf=True,
            )

def run():
    net = Mininet(topo=BasicTopo(), link=TCLink, controller=None, switch=LinuxBridge)
    net.start()

```

```

routes = {
    "R1": [
        ("10.0.4.0/24", "10.1.2.2"),
        ("10.2.3.0/24", "10.1.2.2"),
        ("10.2.4.0/24", "10.1.2.2"),
        ("10.3.4.0/24", "10.1.3.3"),
    ],
    "R2": [
        ("10.0.1.0/24", "10.1.2.1"),
        ("10.0.4.0/24", "10.2.4.4"),
        ("10.1.3.0/24", "10.2.3.3"),
        ("10.3.4.0/24", "10.2.3.4"),
    ],
    "R3": [
        ("10.0.4.0/24", "10.3.4.4"),
        ("10.0.1.0/24", "10.1.3.1"),
        ("10.1.2.0/24", "10.2.3.2"),
        ("10.2.4.0/24", "10.2.3.2"),
    ],
    "R4": [
        ("10.0.1.0/24", "10.2.4.2"),
        ("10.2.3.0/24", "10.2.4.2"),
        ("10.1.2.0/24", "10.2.4.2"),
        ("10.1.3.0/24", "10.3.4.3"),
    ],
}

for router, routing_table in routes.items():
    for subnet, next_hop in routing_table:
        net[router].cmd("ip route add %s via %s" % (subnet, next_hop))

CLI(net)
net.stop()

if __name__ == "__main__":
    setLogLevel("info")
    run()

```

Notice that we are only limiting the bandwidth between routers, but not between hosts and routers.

Traffic Generation and Measurement

We are now ready to test the network to see how fast we can send traffic through it. As an example, we set up the example network depicted in Figure 1

We then use the `iperf` tool to send and measure a flow of traffic from H_1 to H_3 . We first run a server to accept connections on H_3 :

```
# iperf3 -s
```

Then we run the client on H_1 to obtain the following output:

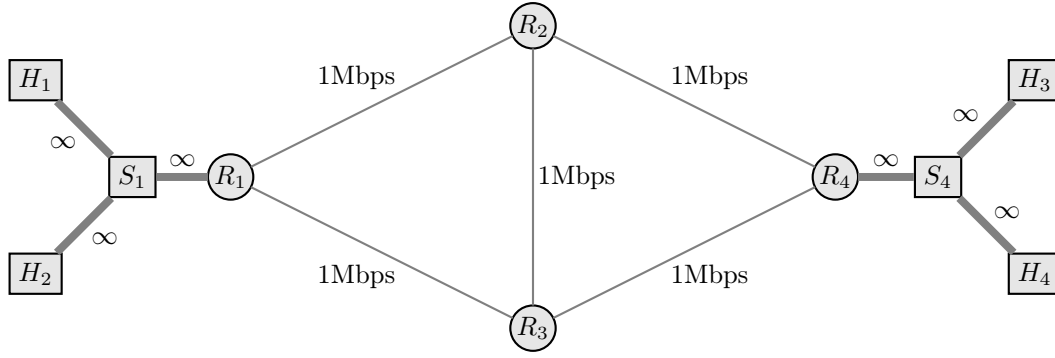


Figure 1: Example Network

```
# iperf3 -c 10.0.4.1
Connecting to host 10.0.4.1, port 5201
[ 5] local 10.0.1.1 port 35466 connected to 10.0.4.1 port 5201
[ ID] Interval           Transfer    Bitrate      Retr  Cwnd
[ 5]  0.00-1.00  sec    256 KBytes  2.10 Mbits/sec  25   24.0 KBytes
[ 5]  1.00-2.00  sec    128 KBytes  1.05 Mbits/sec   5   19.8 KBytes
[ 5]  2.00-3.00  sec    128 KBytes  1.05 Mbits/sec   0   24.0 KBytes
[ 5]  3.00-4.00  sec    128 KBytes  1.05 Mbits/sec   0   25.5 KBytes
[ 5]  4.00-5.00  sec     0.00 Bytes  0.00 bits/sec   3   19.8 KBytes
[ 5]  5.00-6.00  sec    128 KBytes  1.05 Mbits/sec   0   26.9 KBytes
[ 5]  6.00-7.00  sec    128 KBytes  1.05 Mbits/sec   8   24.0 KBytes
[ 5]  7.00-8.00  sec    128 KBytes  1.05 Mbits/sec   1   19.8 KBytes
[ 5]  8.00-9.00  sec    128 KBytes  1.05 Mbits/sec   0   25.5 KBytes
[ 5]  9.00-10.00 sec    128 KBytes  1.05 Mbits/sec   8   21.2 KBytes
- - - - -
[ ID] Interval           Transfer    Bitrate      Retr
[ 5]  0.00-10.00 sec  1.25 MBytes  1.05 Mbits/sec  50
[ 5]  0.00-10.14 sec  1.12 MBytes  931 Kbits/sec

sender
receiver

iperf Done.
```

The performance-analysis tool runs for 10 seconds and prints a throughput report every second. As you can see, the effective bit-rate is a bit under the maximum rate of 1Mbps of each single link. This is because, under the normal routing scheme we use, all the traffic between H_1 and H_3 goes through R_1, R_2, R_4

This performance test transmits data through a single TCP connection, which is subject to the congestion-control mechanism of TCP. Therefore we get similar results using UDP:

```
# iperf3 -c 10.0.4.1 -u
Connecting to host 10.0.4.1, port 5201
[ 5] local 10.0.1.1 port 44816 connected to 10.0.4.1 port 5201
[ ID] Interval           Transfer    Bitrate      Total Datagrams
[ 5]  0.00-1.00  sec    129 KBytes  1.05 Mbits/sec   91
[ 5]  1.00-2.00  sec    127 KBytes  1.04 Mbits/sec   90
[ 5]  2.00-3.00  sec    129 KBytes  1.05 Mbits/sec   91
[ 5]  3.00-4.00  sec    127 KBytes  1.04 Mbits/sec   90
[ 5]  4.00-5.00  sec    129 KBytes  1.05 Mbits/sec   91
```

```

[ 5] 5.00-6.00 sec 129 KBytes 1.05 Mbits/sec 91
[ 5] 6.00-7.00 sec 127 KBytes 1.04 Mbits/sec 90
[ 5] 7.00-8.00 sec 129 KBytes 1.05 Mbits/sec 91
[ 5] 8.00-9.00 sec 127 KBytes 1.04 Mbits/sec 90
[ 5] 9.00-10.00 sec 129 KBytes 1.05 Mbits/sec 91
- - - - -
[ ID] Interval      Transfer    Bitrate      Jitter    Lost/Total Datagrams
[ 5] 0.00-10.00 sec 1.25 MBytes 1.05 Mbits/sec 0.000 ms 0/906 (0%) sender
[ 5] 0.00-10.23 sec 1.20 MBytes 983 Kbits/sec 1.585 ms 38/906 (4.2%) receiver

iperf Done.

```

In this case, the measured bit rates are more stable and higher. Still, the rates are limited by the maximal link rate.

Now, let's try to make a performance test with two parallel TCP connections: one from H_1 to H_3 , and one from H_2 to H_4 . After starting the server on H_4 , we run the client on H_1 :

```

# iperf3 -c 10.0.4.1
Connecting to host 10.0.4.1, port 5201
[ 5] local 10.0.1.1 port 36648 connected to 10.0.4.1 port 5201
[ ID] Interval      Transfer    Bitrate      Retr Cwnd
[ 5] 0.00-1.00 sec 0.00 Bytes 0.00 bits/sec 7 4.24 KBytes
[ 5] 1.00-2.00 sec 0.00 Bytes 0.00 bits/sec 6 8.48 KBytes
[ 5] 2.00-3.00 sec 128 KBytes 1.05 Mbits/sec 0 11.3 KBytes
[ 5] 3.00-4.00 sec 0.00 Bytes 0.00 bits/sec 1 9.90 KBytes
[ 5] 4.00-5.00 sec 128 KBytes 1.05 Mbits/sec 0 12.7 KBytes
[ 5] 5.00-6.00 sec 0.00 Bytes 0.00 bits/sec 0 15.6 KBytes
[ 5] 6.00-7.00 sec 128 KBytes 1.05 Mbits/sec 2 14.1 KBytes
[ 5] 7.00-8.00 sec 128 KBytes 1.05 Mbits/sec 1 14.1 KBytes
[ 5] 8.00-9.00 sec 0.00 Bytes 0.00 bits/sec 0 17.0 KBytes
[ 5] 9.00-10.00 sec 256 KBytes 2.10 Mbits/sec 0 21.2 KBytes
- - - - -
[ ID] Interval      Transfer    Bitrate      Retr
[ 5] 0.00-10.00 sec 768 KBytes 629 Kbits/sec 17 sender
[ 5] 0.00-10.17 sec 640 KBytes 515 Kbits/sec receiver

iperf Done.

```

At the same time we run the client on H_2 :

```

# iperf3 -c 10.0.4.2
Connecting to host 10.0.4.2, port 5201
[ 5] local 10.0.1.2 port 58950 connected to 10.0.4.2 port 5201
[ ID] Interval      Transfer    Bitrate      Retr Cwnd
[ 5] 0.00-1.00 sec 256 KBytes 2.10 Mbits/sec 25 24.0 KBytes
[ 5] 1.00-2.00 sec 128 KBytes 1.05 Mbits/sec 4 19.8 KBytes
[ 5] 2.00-3.00 sec 128 KBytes 1.05 Mbits/sec 1 24.0 KBytes
[ 5] 3.00-4.00 sec 0.00 Bytes 0.00 bits/sec 0 25.5 KBytes
[ 5] 4.00-5.00 sec 128 KBytes 1.05 Mbits/sec 10 12.7 KBytes
[ 5] 5.00-6.00 sec 0.00 Bytes 0.00 bits/sec 0 15.6 KBytes
[ 5] 6.00-7.00 sec 128 KBytes 1.05 Mbits/sec 5 12.7 KBytes
[ 5] 7.00-8.00 sec 0.00 Bytes 0.00 bits/sec 5 12.7 KBytes

```

[5]	8.00-9.00	sec	128 KBytes	1.05 Mbits/sec	4	11.3 KBytes	
[5]	9.00-10.00	sec	0.00 Bytes	0.00 bits/sec	0	14.1 KBytes	

[ID]	Interval		Transfer	Bitrate	Retr		
[5]	0.00-10.00	sec	896 KBytes	734 Kbits/sec	54		sender
[5]	0.00-10.17	sec	768 KBytes	619 Kbits/sec			receiver
iperf Done.							

However, as we should expect by now, the two concurrent connections end-up simply sharing the links, achieving more or less the same total throughput as a single connection.

Max-Flow Routing

One way to break the barrier of the single link rate is to use multiple links. In this case, it is indeed possible to send traffic from H_1 to H_3 through R_2 and the traffic from H_2 to H_4 through R_3 , allowing a higher transmission rate than using a single path. This is illustrated in Figure 2. However, the basic forwarding algorithm would treat every packet going from R_1 to R_4 in exactly the same way. What we need, instead, is a way to selectively route one flow through R_2 , and the other flow rest through R_3 .

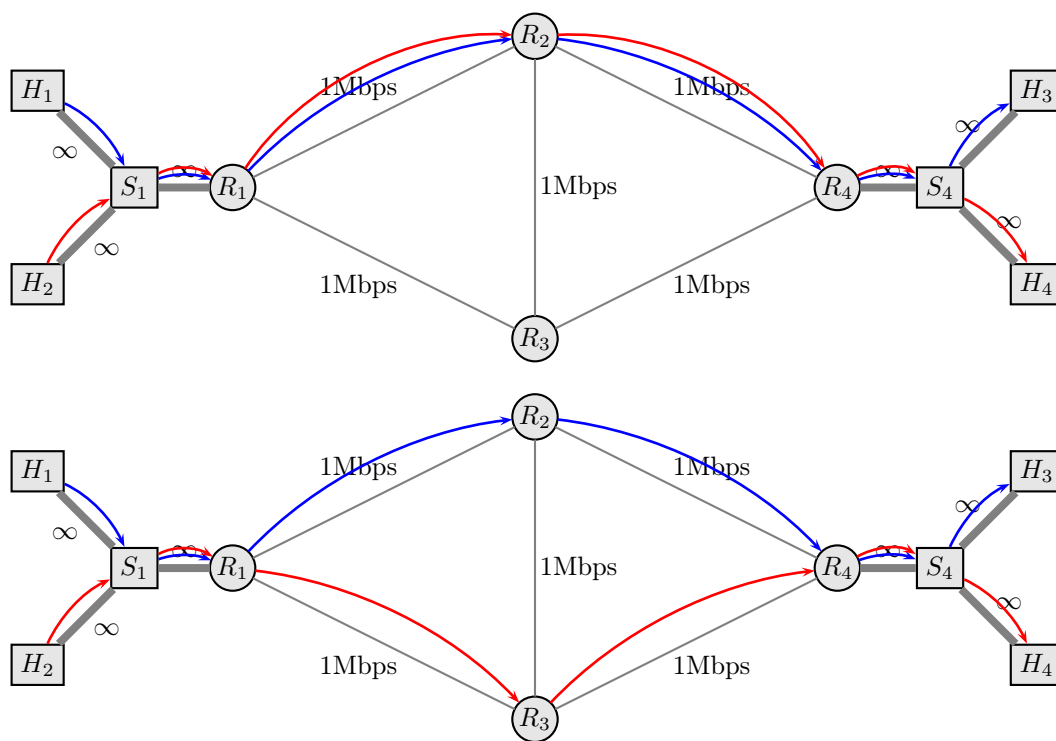


Figure 2: Traditional, Single-Path Routing vs. Multi-Path Whole Flows

Traditional routing based on the shortest path does not accommodate for the two flows to take different paths. A possible way of solving this problem is via *Multi-Protocol Label Switching (MPLS)*. MPLS is a protocol to implement *label switching*. Label switching is a routing technique that moves packets based

on labels attached to IP packets, and not on the destination IP address. Label switching provides several benefits over IP switching. There are several reason for using label switching, but in these notes we will only explore how MPLS can be used to implement the traffic engineering optimizations we have seen.

MPLS Architecture

MPLS does rely on the IP destination address to take its forwarding decisions, but it relies on a fixed-length label added to the packet. Originally, MPLS was introduced to speed up the routing decisions at each hop. Initially, routers had to involve the CPU at each forwarding decision for each hop, and the longest-prefix match is more resource intensive compared to some simple label lookup. However, this is not the case anymore, so if you plan to implement MPLS to speedup packet processing don't.

In MPLS, a label consists of a 32-bit header usually added between the layer 2 and the layer 3 header (often referred as *shim* header). The MPLS header has the following format (you don't need to memorize the header, but having a look at the header helps in understanding how the protocol functions):

Diagram illustrating the 32-bit packet structure:

- 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
- Label | Exp | S | TTL | Stack
- Entry

The first 20 bits consists of the most critical part of the MPLS header, the *Label*. The Label is a number used for taking forwarding decisions. The next 3 bits (Exp) are for experimental use. For now, these three bits are used for Quality of Service, that is they allow packets to be classified and prioritized based on service requirements. MPLS allows to “stack” multiple labels on a packet. The *Bottom of Stack (S)* bit indicates whether this is the last label. If the bit is set to 1, the label is the last (or only) label, and after it’s processed, the payload is handed off to the appropriate Layer 3 protocol (typically IP). If the bit is 0, it indicates that there are additional MPLS labels stacked above, which will be processed before the payload is forwarded. MPLS is not only used for traffic engineering, so there are other applications we won’t cover here such as VPNs for which stacking multiple labels makes sense. An example of such applications is VPNs. Finally, there is *Time to Live (TTL)* field is similar in purpose to the TTL field in an IPv4 header.

A group of routers operating MPLS routing and forwarding is often referred as *MPLS domain*. The routers at the edge of an MPLS domain are often referred as *Label Edge Router (LER)*, while the routers inside the domain are often referred as *Label Switch Router (LSR)*. LSRs are simply routers supporting MPLS forwarding.

When a packet enters an MPLS domain, the router at the edge of the domain is responsible for attaching a label to it. This router is sometimes referred as *MPLS ingress node* or *ingress LER*. The operation of attaching a label to the packet is often referred as *push*. The label assigned by the LER is not random, but it usually depends on some policies configured on the node. After receiving the label the packet is sent into the rest of the MPLS domain.

Inside the domain, the packet will usually go through a series of routers that will forward the packet until it exist the domain. An LSR looks at the label to determine the next hop, and transmits the packet to the next router in the domain. Usually, LSRs also update the label with a new one that is relevant for the next hop forwarding decision. This process is called *label swapping*.

The forwarding process continues until the packet reaches a router at the edge of the MPLS domain. The last router on the path is responsible for removing the label. Then, the packet can be forwarded to its final

destination as if it had just passed through a traditional IP router. The router at the end of the MPLS domain is often referred as *MPLS egress node* or *egress LER*. The operation of removing the label from the packet is referred as *pop*.

One of the key features of MPLS is its ability to create *Label Switched Paths (LSPs)*. These are paths through the network that packets follow based on their labels. In fact, the forwarding decision is taken by the ingress LER which pushes the label on the packet as it enters the MPLS domain. Therefore, MPLS becomes essential in implementing traffic engineering policies since it allows to use and define all the paths obtained from the optimization process.

MPLS Forwarding in Linux

Having a brief introduction to MPLS, let's see how we can implement label switching using MPLS on a Linux machine. Before starting, you need to ensure that your kernel supports MPLS, so you need to ensure that the `mpls_router` and `mpls_iptunnel` kernel modules are loaded. You can check whether they're loaded using the `lsmod` command:

```
# lsmod | grep mpls
mpls_gso          12288  0
mpls_iptunnel     16384  0
mpls_router       49152  1 mpls_iptunnel
ip_tunnel         36864  1 mpls_router
```

If you have an output similar to the one shown above your kernel supports MPLS; otherwise, you have to load those modules with the `modprobe` command:

```
# modprobe mpls_router
# modprobe mpls_iptunnel
```

Notice that for `modprobe` to load the modules you need to be `root`. Now, we need to add some system configuration to allow MPLS processing. The first configuration will set the number of labels allowed. The second configuration controls whether MPLS packets can be received on a given interface. Therefore let's change out initial Mininet script to add these configurations after we install the routing tables:

```
for i in range(1, 5):
    net["R%d" % i].cmd("sysctl -w net.mpls.platform_labels=65535")

for u, v in [(1, 2), (2, 4), (1, 3), (3, 4)]:
    net["R%d" % v].cmd("sysctl -w net.mpls.conf.R%s-eth%d.input=1" % (v, u))
    net["R%d" % u].cmd("sysctl -w net.mpls.conf.R%s-eth%d.input=1" % (u, v))
```

Finally, we can install the MPLS routes using the `ip` utility. In our example, the LERs are R_1 and R_4 . On direct traffic, R_1 is responsible for pushing the label, and R_4 instead will be popping the label. On the return traffic, it will be other way around. Therefore, let's add the rules on the LERs:

```
# push a label depending on the path
net["R1"].cmd("ip route add 10.0.4.1/32 encap mpls 100 via inet 10.1.2.2")
net["R1"].cmd("ip route add 10.0.4.2/32 encap mpls 200 via inet 10.1.3.3")
# pop the label at the egress LSR
net["R4"].cmd("ip -f mpls route add 100 via inet 10.0.4.1")
net["R4"].cmd("ip -f mpls route add 200 via inet 10.0.4.2")

# push a label on the return traffic
net["R4"].cmd("ip route add 10.0.1.1/32 encap mpls 101 via inet 10.2.4.2")
```



```

net["R4"].cmd("ip route add 10.0.1.2/32 encap mpls 201 via inet 10.3.4.3")
# pop the label at the egress LSR
net["R1"].cmd("ip -f mpls route add 101 via inet 10.0.1.1")
net["R1"].cmd("ip -f mpls route add 201 via inet 10.0.1.2")

```

Now, we will add the routes on the LSRs. In this implementation, we will swap the label with the same one received, but we do not have to. In fact, labels are local to the single router. The code to add the forwarding rules on the LSRs is the following:

```

# adding label swapping rules on R2
net["R2"].cmd("ip -f mpls route add 100 as 100 via inet 10.2.4.4")
net["R2"].cmd("ip -f mpls route add 101 as 101 via inet 10.1.2.1")
# adding label swapping rules on R3
net["R3"].cmd("ip -f mpls route add 200 as 200 via inet 10.3.4.4")
net["R3"].cmd("ip -f mpls route add 201 as 201 via inet 10.1.3.1")

```

Now, if we try the same performance test, we will be able to get a better throughput for both flows. On H_1 , the output will be something similar to the following:

```

# iperf3 -c 10.0.4.1
Connecting to host 10.0.4.1, port 5201
[ 5] local 10.0.1.1 port 53708 connected to 10.0.4.1 port 5201
[ ID] Interval          Transfer    Bitrate      Retr  Cwnd
[ 5]  0.00-1.00 sec    256 KBytes 2.10 Mbits/sec 25   24.0 KBytes
[ 5]  1.00-2.00 sec    128 KBytes 1.05 Mbits/sec  4   19.7 KBytes
[ 5]  2.00-3.00 sec    128 KBytes 1.05 Mbits/sec  1   24.0 KBytes
[ 5]  3.00-4.00 sec    128 KBytes 1.05 Mbits/sec  0   25.4 KBytes
[ 5]  4.00-5.00 sec     0.00 Bytes 0.00 bits/sec  3   19.7 KBytes
[ 5]  5.00-6.00 sec    128 KBytes 1.05 Mbits/sec  0   26.8 KBytes
[ 5]  6.00-7.00 sec    128 KBytes 1.05 Mbits/sec  1   22.6 KBytes
[ 5]  7.00-8.00 sec    128 KBytes 1.05 Mbits/sec  8   16.9 KBytes
[ 5]  8.00-9.00 sec    128 KBytes 1.05 Mbits/sec  0   25.4 KBytes
[ 5]  9.00-10.00 sec   128 KBytes 1.05 Mbits/sec  2   19.7 KBytes
- - - - -
[ ID] Interval          Transfer    Bitrate      Retr
[ 5]  0.00-10.00 sec   1.25 MBytes 1.05 Mbits/sec 44
[ 5]  0.00-10.16 sec   1.12 MBytes 929 Kbits/sec
                                sender
                                receiver

iperf Done.

```

If we run the performance test on H_2 at the same time, we will obtain an output similar to the following:

```

# iperf3 -c 10.0.4.2
Connecting to host 10.0.4.2, port 5201
[ 5] local 10.0.1.2 port 50732 connected to 10.0.4.2 port 5201
[ ID] Interval          Transfer    Bitrate      Retr  Cwnd
[ 5]  0.00-1.00 sec    256 KBytes 2.10 Mbits/sec 25   24.0 KBytes
[ 5]  1.00-2.00 sec    128 KBytes 1.05 Mbits/sec  5   19.7 KBytes
[ 5]  2.00-3.00 sec    128 KBytes 1.05 Mbits/sec  0   24.0 KBytes
[ 5]  3.00-4.00 sec    128 KBytes 1.05 Mbits/sec  0   25.4 KBytes
[ 5]  4.00-5.00 sec     0.00 Bytes 0.00 bits/sec  3   19.7 KBytes
[ 5]  5.00-6.00 sec    128 KBytes 1.05 Mbits/sec  0   26.8 KBytes
[ 5]  6.00-7.00 sec    128 KBytes 1.05 Mbits/sec  8   24.0 KBytes

```

```

[ 5]  7.00-8.00  sec  128 KBytes 1.05 Mbits/sec  1  19.7 KBytes
[ 5]  8.00-9.00  sec  128 KBytes 1.05 Mbits/sec  0  25.4 KBytes
[ 5]  9.00-10.00 sec  128 KBytes 1.05 Mbits/sec  8  22.6 KBytes
- - - - -
[ ID] Interval      Transfer    Bitrate      Retr
[ 5]  0.00-10.00 sec  1.25 MBytes 1.05 Mbits/sec 50
[ 5]  0.00-10.19 sec  1.12 MBytes 926 Kbits/sec
                                sender
                                receiver

iperf Done.

```